Day 9: Embeddings and Vector Databases for RAG

5-line beginner summary

1. **Embeddings** convert text into numbers so computers can understand meaning.
2. Similar meaning texts get similar number patterns, even if words are different.
3. A **vector database** stores these embeddings and helps search by meaning.
4. In RAG, vector databases help find the most relevant document chunks for a user question.
5. The LLM then uses those retrieved chunks to generate a grounded answer.

---

1. What embeddings are

An **embedding** is a numerical representation of text.

Instead of storing text only as words, we convert it into a list of numbers called a **vector**.

Example:

```
Text:
"Employees can apply for maternity leave for 26 weeks."

Embedding:
[0.12, -0.45, 0.88, 0.31, ...]
```

This vector captures the **meaning** of the text.

So, these two sentences may have similar embeddings:

```
"Employees can apply for maternity leave."
"Staff members are eligible for maternity leave."
```

Even though the words are different, the meaning is similar.

---

2. Why text is converted into numbers

Computers and machine learning models work better with numbers than raw text.

Traditional search checks exact words.

Example query:

```
"What is the maternity leave policy?"
```

A keyword search may look for:

```
maternity
leave
policy
```

But what if the document says:

```
"Female employees are eligible for 26 weeks of paid childbirth-related absence."
```

There is no exact phrase “maternity leave policy”, but the meaning is related.

Embeddings solve this problem because they compare **meaning**, not only exact words.

---

3. Sentence embeddings

A **sentence embedding** represents the meaning of a complete sentence or paragraph.

Example:

```
Sentence 1:
"Employees can work from home two days per week."

Sentence 2:
"Staff are allowed remote work twice weekly."

Sentence 3:
"Employees must submit travel bills within 30 days."
```

Sentence 1 and Sentence 2 should have similar embeddings because they mean similar things.

Sentence 3 should have a different embedding because it is about travel reimbursement.

In RAG systems, we usually do not embed full documents directly. We split documents into smaller chunks and embed each chunk.

Example:

```
Policy Document
    ↓
Chunk 1: Leave policy
Chunk 2: Work from home policy
Chunk 3: Travel reimbursement policy
    ↓
Embeddings for each chunk
```

---

4. Similarity search

Similarity search means finding text chunks whose embeddings are closest to the user query embedding.

Example user question:

```
"How many days of maternity leave are allowed?"
```

The system converts this question into an embedding.

Then it searches the vector database for chunks with similar embeddings.

Possible retrieved chunk:

```
"Female employees are eligible for 26 weeks of paid maternity leave."
```

This is useful because the system finds relevant content by meaning.

---

5. Cosine similarity

Cosine similarity is a common method to compare two vectors.

It checks whether two vectors point in a similar direction.

Simple idea:

```
High cosine similarity   = similar meaning
Low cosine similarity    = different meaning
```

Example:

```
Query:
"What is the leave policy for new mothers?"

Chunk A:
"Employees can take 26 weeks of maternity leave."
Similarity score: 0.91

Chunk B:
"Employees must submit travel expenses within 30 days."
Similarity score: 0.23
```

Chunk A is more relevant because the score is higher.

Cosine similarity usually ranges from:

```
-1 to 1
```

In many embedding search systems, a score closer to **1** means more similar.

---

6. Vector database purpose

A **vector database** stores embeddings and allows fast similarity search.

It usually stores:

```
1. Text chunk
2. Embedding vector
3. Metadata
4. Document source
5. Page number or section name
```

Example record:

```
{
  "chunk_id": "policy_leave_001",
  "text": "Employees can take 26 weeks of maternity leave.",
```

```
  "embedding": [0.12, -0.45, 0.88, ...],
  "metadata": {
    "document_name": "HR Policy 2025",
    "department": "HR",
    "page": 12,
    "policy_type": "leave"
  }
}
```

The vector database helps answer questions like:

```
Find the top 5 chunks most similar to this user question.
```

---

7. FAISS

FAISS stands for Facebook AI Similarity Search.

It is a library used for fast vector search.

Important points:

```
FAISS is not a full enterprise database by itself.
It is mainly a vector search library.
It is useful for local experiments and high-speed similarity search.
```

Good for:

```
- Local RAG prototypes
- Fast similarity search
- Research and experimentation
- When you want control over indexing
```

Example use:

```
Use FAISS to store embeddings locally and retrieve similar chunks for a chatbot.
```

Simple view:

```
Documents → Chunks → Embeddings → FAISS Index → Top-k Results
```

---

8. Chroma

Chroma is a developer-friendly vector database often used in RAG prototypes.

Good for:

```
- Learning RAG
- Local development
- Quick proof of concept
- Integration with LangChain and LlamaIndex
```

Chroma can store:

```
- Embeddings
- Text chunks
- Metadata
- Collection names
```

Example:

```
Collection: "hr_policy_docs"

Chunk:
"Employees can work remotely twice per week."

Metadata:
{
  "department": "HR",
  "document": "Work From Home Policy"
}
```

Chroma is usually easier for beginners than managing FAISS manually.

---

9. Pinecone

Pinecone is a managed vector database service.

Managed means the platform handles much of the infrastructure for you.

Good for:

```
- Production RAG applications
- Scalable vector search
- Cloud-based applications
- Teams that do not want to manage vector DB servers manually
```

Typical enterprise use:

```
Company documents → Embeddings → Pinecone → RAG chatbot
```

Pinecone is commonly used when the team wants a hosted vector search solution with APIs.

---

10. Weaviate

Weaviate is an open-source vector database with strong semantic search capabilities.

Good for:

```
- Semantic search
- Hybrid search
- Metadata filtering
- Enterprise RAG
- Self-hosted or managed deployment
```

Hybrid search means combining:

```
Keyword search + Vector search
```

Example:

```
Search for "maternity leave" using both exact words and semantic meaning.
```

This can improve retrieval quality because sometimes keyword matching is still useful.

---

11. Milvus

Milvus is an open-source vector database designed for large-scale vector search.

Good for:

```
- Large embedding collections
- High-performance similarity search
- Enterprise-scale AI systems
- Self-hosted vector database deployments
```

Milvus is often used when organizations need to manage large volumes of vectors and want more control over infrastructure.

Example:

```
A company has 50 million document chunks.
Milvus can be used to store and search those vectors efficiently.
```

---

12. Metadata filtering

Metadata filtering means narrowing search results using extra information.

Example document chunks:

```
Chunk 1:
Text: "Employees can apply for maternity leave."
Metadata: department = HR, country = India

Chunk 2:
Text: "Employees can apply for parental leave."
Metadata: department = HR, country = USA

Chunk 3:
Text: "Employees can claim travel reimbursement."
Metadata: department = Finance, country = India
```

User asks:

```
"What is the maternity leave policy in India?"
```

The vector DB can search only where:

```
country = India
department = HR
```

This improves accuracy.

Without metadata filtering, the system may retrieve policy chunks from the wrong country, department, or year.

Common metadata fields:

```
document_name
department
country
business_unit
year
version
page_number
access_level
created_date
policy_type
```

Metadata filtering is very important in enterprise RAG because different users may have access to different documents.

---

13. Top-k retrieval

Top-k retrieval means retrieving the top `k` most relevant chunks.

Example:

```
k = 3
```

The vector database returns the top 3 most similar chunks.

Example query:

```
"What is the work from home policy?"
```

Results:

```
Top 1: Work from home allowed two days per week.
Top 2: Manager approval is required for remote work.
Top 3: Remote work is not allowed during probation.
```

Choosing `k` is important.

If `k` is too small:

```
The system may miss important context.
```

If `k` is too large:

```
The prompt may become noisy and expensive.
```

Typical starting values:

```
top_k = 3
top_k = 5
top_k = 10
```

For interview or POC, starting with `top_k = 3` or `top_k = 5` is usually easy to explain.

---

14. How vector DB fits into enterprise RAG

In enterprise RAG, the vector database acts like a **semantic memory layer**.

It stores searchable knowledge from enterprise documents.

Example documents:

```
HR policies
IT support documents
Insurance documents
Banking policies
Legal guidelines
Product manuals
SOP documents
Project documentation
```

RAG flow:

```
User asks question
    ↓
Question is converted into embedding
    ↓
Vector DB searches relevant chunks
    ↓
Top chunks are added to prompt
    ↓
LLM generates answer using retrieved context
    ↓
Answer includes source citations
```

Without a vector database, the LLM only depends on its trained knowledge.

With a vector database, the LLM can answer using company-specific documents.

Easy example using policy documents

Imagine a company has these policy documents:

```
1. HR Leave Policy
2. Work From Home Policy
3. Travel Reimbursement Policy
4. Laptop Allocation Policy
5. Employee Insurance Policy
```

A user asks:

```
"Can I work from home during probation?"
```

The system should not search only by exact words. It should understand meaning.

Step-by-step:

```
Step 1:
Split policy documents into chunks.

Step 2:
Convert each chunk into an embedding.

Step 3:
Store embeddings in vector DB with metadata.

Step 4:
Convert user question into embedding.

Step 5:
Search similar chunks.

Step 6:
Retrieve top-k chunks.

Step 7:
Send chunks to LLM.

Step 8:
LLM answers using only retrieved policy content.
```

Possible retrieved chunk:

```
"Employees under probation are not eligible for regular work from home unless
approved by their manager."
```

Final answer:

```
As per the Work From Home Policy, employees under probation are generally not
eligible for regular work from home unless they receive manager approval.
```

This answer is better because it is grounded in company policy.

ASCII diagram: Embedding and search flow

```
                DOCUMENT INDEXING FLOW
                ----------------------

    Company Documents
    HR Policy, IT Policy, Travel Policy
               |
               v
```

```
       Document Loader
              |
              v
       Text Extraction
              |
              v
         Chunking
  small meaningful text pieces
              |
              v
      Embedding Model
converts chunks into vectors
              |
              v
       Vector Database
stores vectors + text + metadata


               QUESTION ANSWERING FLOW
               -----------------------

          User Question
"Can I work from home during probation?"
              |
              v
       Embedding Model
converts question into vector
              |
              v
       Vector Database Search
similarity search + metadata filter
              |
              v
         Top-k Chunks
most relevant policy sections
              |
              v
      Prompt Construction
question + retrieved context
              |
              v
             LLM
generates grounded answer
              |
              v
       Final Answer + Citations
```

Pseudocode for indexing documents

```
# Step 1: Load documents
documents = load_documents(folder_path="company_policy_docs/")
```

```
# Step 2: Split documents into smaller chunks
chunks = []

for document in documents:
    document_chunks = split_text(
        text=document.text,
        chunk_size=500,
        chunk_overlap=50
    )

    for chunk in document_chunks:
        chunks.append({
            "text": chunk,
            "metadata": {
                "document_name": document.name,
                "department": document.department,
                "page_number": document.page_number,
                "access_level": document.access_level
            }
        })

# Step 3: Convert each chunk into embedding
for chunk in chunks:
    chunk["embedding"] = embedding_model.embed(chunk["text"])

# Step 4: Store chunks in vector database
for chunk in chunks:
    vector_db.insert(
        id=generate_unique_id(),
        vector=chunk["embedding"],
        text=chunk["text"],
        metadata=chunk["metadata"]
    )

print("Document indexing completed successfully.")
```

Pseudocode for retrieving top-k chunks

```
# User question
user_question = "Can I work from home during probation?"

# Step 1: Convert user question into embedding
query_embedding = embedding_model.embed(user_question)

# Step 2: Define metadata filter
metadata_filter = {
    "department": "HR",
    "document_type": "policy"
}

# Step 3: Search vector database
top_k_results = vector_db.search(
```

```
    query_vector=query_embedding,
    top_k=5,
    filter=metadata_filter
)

# Step 4: Extract retrieved text chunks
retrieved_context = []

for result in top_k_results:
    retrieved_context.append({
        "text": result.text,
        "source": result.metadata["document_name"],
        "page": result.metadata["page_number"],
        "score": result.similarity_score
    })

# Step 5: Build prompt for LLM
prompt = build_prompt(
    question=user_question,
    context=retrieved_context
)

# Step 6: Generate answer
answer = llm.generate(prompt)

# Step 7: Return answer with sources
return {
    "answer": answer,
    "sources": retrieved_context
}
```

Simple comparison table

Tool	What it is	Best for	Simple explanation
FAISS	Vector search library	Local search, experiments, fast indexing	Good for building custom vector search
Chroma	Vector database	RAG POCs and local apps	Beginner-friendly vector DB
Pinecone	Managed vector DB	Cloud production RAG	Hosted service for scalable vector search
Weaviate	Open-source vector DB	Semantic and hybrid search	Combines vector and keyword search well
Milvus	Open-source vector DB	Large-scale enterprise search	Strong option for huge vector collections

---

How this connects to IBM AI/GenAI roles

For IBM AI/GenAI roles, you should be able to explain this clearly:

```
In enterprise RAG, documents are first ingested, cleaned, chunked, embedded, and
stored in a vector database.

When a user asks a question, the same embedding model converts the question into
a vector. The vector database retrieves the most relevant chunks using
similarity search and metadata filters. These chunks are passed to the LLM as
context so the answer is grounded in enterprise documents instead of only model
memory.
```

Interview-friendly answer:

```
A vector database is important in RAG because it allows semantic search over
enterprise knowledge. Instead of relying on keyword matching, it retrieves
chunks based on meaning. This helps the LLM answer questions using relevant
internal documents, reduces hallucination, supports citations, and allows
filtering by metadata such as department, country, document version, and access
level.
```

Common mistakes

1. Using very large chunks

Bad:

```
One full 20-page policy document as one chunk.
```

Problem:

```
The retrieved context may be too broad and noisy.
```

Better:

```
Split into smaller meaningful chunks.
```

---

2. Using very tiny chunks

Bad:

```
"Employees are eligible."
```

Problem:

```
This does not contain enough meaning.
```

Better:

```
"Employees are eligible for 26 weeks of maternity leave after completing 6
months of service."
```

3. Not using metadata

Bad:

```
Search all documents without filters.
```

Problem:

```
The system may retrieve outdated or wrong-country policy.
```

Better:

```
Filter by country, department, year, document version, and access level.
```

---

4. Retrieving too many chunks

Bad:

```
top_k = 50
```

Problem:

```
The LLM gets too much irrelevant context.
```

Better:

```
Start with top_k = 3 or top_k = 5.
```

---

5. Retrieving too few chunks

Bad:

```
top_k = 1
```

Problem:

```
The answer may miss important details.
```

Better:

```
Use top_k = 3 to 5 and evaluate the answer quality.
```

---

6. Using different embedding models for indexing and querying

Bad:

```
Index documents with Model A.
Search user question with Model B.
```

Problem:

```
Vectors may not be comparable.
```

Better:

```
Use the same embedding model for both indexing and querying.
```

---

7. Not storing source information

Bad:

```
Store only text and embedding.
```

Problem:

```
The system cannot show citations.
```

Better:

```
Store document name, page number, section, and version.
```

---

8. Thinking vector DB removes hallucination completely

Vector databases reduce hallucination, but they do not completely remove it.

The LLM may still make mistakes if:

```
Wrong chunks are retrieved
Documents are outdated
Prompt is poorly written
The model ignores context
Metadata filtering is missing
```

A good RAG system needs:

```
Good chunking
Good embeddings
Good retrieval
Good metadata
Good prompt design
Good evaluation
```

Final mental model

Think of embeddings and vector databases like this:

```
Embeddings = Meaning converted into numbers

Vector Database = Search engine for meaning

RAG = LLM + relevant enterprise knowledge
```

Simple example:

```
User asks:
"What is the maternity leave policy?"

Vector DB finds:
The maternity leave section from HR policy.

LLM answers:
Based on the retrieved HR policy, employees are eligible for 26 weeks of
maternity leave.
```

That is the core idea of embeddings and vector databases in enterprise RAG.